

Assignment # 1

Generative AI

Deadline: March 16, 2024

In this assignment, you will design, train, evaluate, and deploy four related generative AI systems. The first three tasks concern image restoration using autoencoders, adaptive routing, and a soft mixture-of-experts architecture. The fourth task concerns paired image-to-image generation using a conditional generative adversarial network.

This is both an implementation-based and research-oriented assignment. You are expected to independently investigate concepts, techniques, tools, and implementation challenges that have not been completely covered during the lectures. You must consult credible research papers, official documentation, technical resources, and appropriate open-source implementations to understand the available solutions. You should investigate alternative architectures, loss functions, training strategies, hyperparameter ranges, evaluation methods, and deployment approaches before making your final design decisions. The purpose is not only to produce working models but also to develop the ability to identify what you do not know, locate reliable information, learn unfamiliar technologies, and apply that knowledge to solve a practical problem.

Every important technical decision must be supported by evidence. Your report should explain what alternatives you investigated, why a particular approach was selected, what difficulties were encountered, and how research or experimentation helped you address those difficulties. Simply reproducing an existing implementation or accepting AI-generated code without investigation will not satisfy the research component. You are expected to demonstrate independent reasoning, experimentation, critical analysis, and learning beyond the material directly presented in class.

This assignment must be completed individually. You may use generative AI assistants, coding assistants, pretrained software libraries, research tools, and relevant open-source resources. However, you must verify all generated information and code, test every component, acknowledge reused material, and understand the complete submitted system. You remain responsible for the correctness, originality, and reliability of your work. During the evaluation, you may be asked to justify your architecture, explain a research-informed decision, interpret an experimental result, modify part of the implementation, or execute the system using previously unseen images.

The assignment will not be considered complete if the models can only be executed from a notebook, command line, or development environment. All four tasks must be integrated into a properly designed browser-based or mobile application through which an evaluator can provide an input, execute the selected model, inspect the generated output, and observe relevant system information. The final submission must therefore demonstrate the complete process from research and model development to evaluation, integration, and deployment as a functional software product.

Instructions

- This is an individual assignment and must be submitted through Google Classroom no later than the given deadline. Late submissions will not be accepted in any case.
- Submit a technical report written in the format of a research paper in IEEE format only via Latex: The report must discuss each of the four tasks separately and model architecture, loss functions, training procedure, hyperparameter optimization, experimental results, visual outputs, failure cases, application design, and conclusions. Architecture diagrams, training graphs, confusion

matrices, routing visualizations, generated images, error maps, result tables, and application screenshots must be included where relevant.

IEEE Paper Format

- Maintain the complete implementation in a GitHub repository. In the technical report include the link to the repository URL. The repository must contain the source code, configuration files, dependency file, data-preparation scripts, training scripts, evaluation scripts, Optuna studies, ONNX export code, application code, Dockerfiles, Docker Compose configuration, and a README containing complete execution instructions. Do not upload the complete datasets or very large model files directly to GitHub. Provide a documented download link or use Git LFS for the trained models.
- Use PyTorch or tensorflow or any suitable language to implement and train all models. Optuna must be used for hyperparameter optimization in each of the four tasks. Important training experiments, hyperparameters, losses, evaluation results, checkpoints, and visual outputs must be recorded using either MLflow or Weights & Biases.
- Design the application interface using Google Stitch and include evidence of the original Stitch design in the report. Implement the frontend using React and Tailwind CSS and the backend using services such as FastAPI. All four tasks must be accessible through a single browser-based application.
- Export the trained models required for inference to ONNX. Verify that the ONNX outputs remain consistent with their corresponding PyTorch / tensorflow or any other used language outputs. Include the ONNX models or provide valid download links for them.
- The frontend and backend must run inside Docker containers. Provide a Docker Compose configuration that starts the complete application with one documented command. The evaluator must be able to clone the repository, obtain the model files, start the containers, and access the application through a browser without using VS Code or manually running separate Python scripts.
- Record a five-to-seven-minute demonstration video showing the application startup process and the successful operation of all four tasks and upload this video to YouTube and include only the link in the report to your YouTube video. The demonstration must show image uploading, runtime corruption, universal restoration, hard routing, soft expert weights, face-to-sketch generation, result downloading, and experiment-tracking records. You must not upload video directly to Google Classroom.
- AI-assisted tools may be used for research, coding, interface design, debugging, and documentation. However, the student remains responsible for verifying the generated material and understanding the complete implementation. Include an AI-use appendix identifying the tools used, the tasks for which they were used, and how their outputs were tested or corrected. During the evaluation, students may be asked to explain or modify any submitted component and run the application on previously unseen images.

Dataset for Tasks 1–3

Tasks 1, 2, and 3 will use the [Oxford-IIIT Pet Dataset](#). The dataset contains images of cats and dogs belonging to 37 categories. The category labels are not required for the main restoration problem; the original images will serve as the clean targets.

Use the official training and validation collection as the development data. Divide it into 80% training and 20% validation data using random seed 42. The official test set must remain untouched

until final evaluation. All images should be converted to RGB and resized to 128×128 pixels. The same data split must be used throughout Tasks 1, 2, and 3.

The dataset contains clean images. Therefore, the corrupted inputs must be generated programmatically. During training, corruption must be applied at runtime inside the data-loading pipeline. A new corruption type and severity should be sampled whenever an image is loaded. You should not permanently save thousands of corrupted copies of the dataset.

For every clean training image, the data loader must randomly select one of four possible input conditions with equal probability: clean, salt-and-pepper noise, Gaussian blur, or rectangular occlusion. The selected corruption label is known when it is generated and will later be used to train the corruption classifier.

The following corruption definitions must be used.

Input condition	Training configuration
Clean	The original image is provided without artificial corruption.
Salt-and-pepper noise	Sample a corruption probability uniformly between 0.02 and 0.15. Replace the selected pixels with black or white values using equal probability.
Gaussian blur	Select a kernel size from 3, 5, or 7 and sample the Gaussian standard deviation uniformly between 0.5 and 2.5.
Rectangular occlusion	Insert between one and three black rectangular masks. The masks must jointly cover between 10% and 35% of the image area. Their locations must be sampled randomly.

Training corruptions should be produced dynamically. Validation and test corruptions, however, must be deterministic. Generate a validation corruption manifest and a test corruption manifest once and store the corruption type, severity, mask coordinates, blur settings, and random seed associated with every image.

For final testing, each clean test image must have three fixed severity levels for every corruption. Salt-and-pepper probabilities must be 0.03, 0.08, and 0.15. Gaussian blur must use the configurations (3, 0.7), (5, 1.5), and (7, 2.5), where each pair represents kernel size and standard deviation. Occlusion must cover approximately 10%, 20%, and 35% of the image using one, two, and three rectangles, respectively.

Task 1: Universal Multi-Corruption Denoising Autoencoder

In the first task, you must design a single universal denoising autoencoder capable of processing clean images and images affected by any of the three specified corruptions. The model will not be told which corruption has been applied. It must learn one shared representation that can reconstruct the clean target from all supported input conditions.

The network must contain a convolutional encoder, a genuine compressed latent representation, and a convolutional decoder. The encoder should progressively reduce the spatial dimensions while increasing the number of channels. The decoder should reconstruct an RGB image of the original resolution. The network must contain a meaningful bottleneck; simply copying the input through

unrestricted skip connections will not satisfy the autoencoder requirement. If limited skip connections are used, their purpose and effect must be investigated and justified in the report.

For every training example, let x represent the clean image, $\tilde{x}$ the corrupted input, and $\hat{x}$ the reconstructed image:

$$\hat{x} = D(E(\tilde{x}))$$

Here, E is the encoder and D is the decoder. The model should be trained using a combination of pixel reconstruction loss and structural similarity loss:

$$\mathcal{L}_{UDAE} = \alpha \mathcal{L}_{L1}(x, \hat{x}) + (1 - \alpha)(1 - \text{SSIM}(x, \hat{x}))$$

The L1 component encourages accurate pixel reconstruction, while the Structural Similarity Index Measure (SSIM) component encourages the reconstructed image to preserve important structures, shapes, edges, and local contrast. A reasonable initial value is $\alpha = 0.8$, but the final value must be selected through Optuna rather than being accepted without investigation.

The Optuna study for this task must investigate at least the learning rate, batch size, bottleneck dimension, number of encoder channels, dropout rate, and loss-weight value α . The validation objective should combine reconstruction quality and structural similarity. The complete search space, number of completed trials, best trial, and final selected configuration must be reported.

The final results for this task must separately show performance on clean images, salt-and-pepper noise, Gaussian blur, and occluded images. Results must also be separated by low, medium, and high corruption severity. Visual results must show the clean target, corrupted input, reconstructed output, and an absolute error map. At least twelve representative examples and four meaningful failure cases must be discussed.

The final model must be exported to ONNX and made available through the application under a workspace named Universal Restoration. A user should be able to upload a corrupted image or select a clean sample and apply one of the available corruptions through the interface. The application should display the input, restored output, selected corruption settings, and inference time.

Task 2: Corruption Classification and Hard-Routed Specialist Autoencoders

Create a hard-routing restoration system. Instead of requiring one autoencoder to learn every restoration problem, this system will use a corruption classifier followed by specialist autoencoders.

First, train a convolutional classifier that predicts one of four input classes: clean, salt-and-pepper noise, Gaussian blur, or rectangular occlusion. The training labels are obtained automatically from the runtime corruption pipeline. The training batches must be balanced so that the classifier does not become biased toward one corruption class.

For a corrupted image $\tilde{x}$, the classifier should produce four class probabilities:

$$p = C(\tilde{x}) = [p_{\text{clean}}, p_{\text{salt}}, p_{\text{blur}}, p_{\text{occlusion}}]$$

The predicted corruption class is the class with the maximum probability:

$$r = \arg \max_k p_k$$

The classifier must be trained using multiclass cross-entropy loss. Optuna must be used to tune its learning rate, batch size, convolutional channel configuration, dropout rate, and weight decay. Its results must include overall accuracy, macro-averaged precision, recall and F1-score, per-class measurements, and a normalized four-class confusion matrix.

After training the classifier, train three specialist denoising autoencoders. One specialist should restore salt-and-pepper images, one should restore blurred images, and one should restore occluded images. Each specialist should be trained only on the type of corruption for which it is responsible. The same clean target image must be used as the reconstruction target.

The specialists may use the same basic architecture, but they must have independently trained parameters. Optuna should be applied to the specialist models. To keep the experiment computationally feasible, you may use a shared Optuna search to identify a common architecture and then train the three specialists independently using that architecture. You should tune the learning rate, bottleneck size, channel configuration, batch size, and L1-to-SSIM loss weighting.

During hard-routed inference, the classifier must first determine the corruption type. The selected specialist must then restore the image:

$$\hat{x} = \begin{cases} \tilde{x}, & r = \text{clean} \\ A_{\text{salt}}(\tilde{x}), & r = \text{salt} \\ A_{\text{blur}}(\tilde{x}), & r = \text{blur} \\ A_{\text{occlusion}}(\tilde{x}), & r = \text{occlusion} \end{cases}$$

A clean input must use an identity bypass and should not be unnecessarily processed by a restoration expert.

The system must be tested in two modes. In oracle-routing mode, the known corruption label from the deterministic test manifest should select the expert. In predicted-routing mode, the classifier prediction should select the expert. Oracle routing shows the restoration ability of the specialists, while predicted routing shows the performance of the complete operational system. Cases in which classifier errors cause restoration failures must be identified and discussed.

The application must provide a separate workspace named Hard-Routed Restoration. It should display the four classifier probabilities, the predicted corruption, the selected expert, the reconstructed image, and inference time. The classifier and all three specialists must be exported to ONNX.

Task 3: Jointly Trained Soft Mixture-of-Experts Restoration

Hard routing sends an image to only one specialist. This may fail when an image contains more than one corruption, when the classifier is uncertain, or when the corruption lies near the boundary between two classes. In this task, you must transform the hard-routing system into a differentiable soft mixture-of-experts model.

The soft system should contain a gating network and the three specialist restoration experts developed in Task 2. It must also contain an identity branch for clean images. Instead of selecting only one branch, the gating network must assign a continuous weight to every branch.

For input $\tilde{x}$, the gating network should calculate:

$$w = \text{softmax}\left(\frac{G(\tilde{x})}{\tau}\right)$$

Here, $w = [w_0, w_1, w_2, w_3]$ contains the weights for the clean identity branch, salt-and-pepper expert, blur expert, and occlusion expert. The temperature τ controls whether the routing weights are sharp or distributed.

The final reconstruction should be calculated as:

$$\hat{x} = w_0\tilde{x} + w_1A_{\text{salt}}(\tilde{x}) + w_2A_{\text{blur}}(\tilde{x}) + w_3A_{\text{occlusion}}(\tilde{x})$$

This weighted combination is differentiable. Therefore, the gate and experts can be trained jointly through the final reconstruction error.

You must not begin joint training with completely random components. Initialize the gating network using the trained classifier from Task 2 and initialize the experts using the three trained specialist autoencoders. Begin with a short warm-up stage in which the experts are frozen and only the gate is trained. After the warm-up, unfreeze the experts and jointly fine-tune the complete system using a smaller learning rate.

The joint loss should contain reconstruction loss, structural similarity loss, corruption-classification loss, and a routing-balance regularizer:

$$\mathcal{L}_{MSE} = \lambda_1\mathcal{L}_{L1} + \lambda_s(1 - \text{SSIM}) + \lambda_c\mathcal{L}_{CE} + \lambda_b\mathcal{L}_{balance}$$

The cross-entropy component keeps the gate related to the known runtime corruption label. The balance component prevents the system from sending nearly every input to the same expert. One possible balance loss is:

$$\mathcal{L}_{balance} = \sum_{k=0}^3 \left(\bar{w}_k - \frac{1}{4}\right)^2$$

In this expression, $\bar{w}_k$ is the average routing weight assigned to branch k within a balanced training batch. You may propose a different differentiable balance or entropy regularizer, but it must be supported by research and clearly justified.

You may begin with $\lambda_1 = 0.8$, $\lambda_s = 0.2$, $\lambda_c = 0.1$, and $\lambda_b = 0.01$. Optuna must then be used to investigate the joint fine-tuning learning rate, temperature τ , classification weight, balance weight, and the reconstruction-loss weighting. Trial pruning may be used when a configuration performs poorly or exhibits routing collapse.

In addition to reconstruction results, you must examine the behaviour of the gating network. Report the average expert weights for every true corruption type and severity level. Show examples in which one expert dominates and examples in which weights are distributed across multiple experts. A routing heatmap or weight-distribution diagram must be included in the technical report. You must also determine whether any expert has become inactive or whether one expert dominates unrelated inputs.

The application must provide a workspace named Soft Mixture-of-Experts Restoration. It should display all four routing weights, the reconstructed result, inference time, and a visual indication of which experts contributed most strongly. The complete soft mixture-of-experts inference pipeline must be exported to ONNX.

Task 4: Style-Conditioned Face-to-Sketch Generation Using a Conditional GAN

For the fourth task, you must build a face-to-sketch generation system using a conditional GAN. Use the [FS2K Facial Sketch Synthesis Dataset](#), which contains 2,104 paired facial photographs and sketches with three sketch-style categories.

Use the official FS2K training and testing definitions. Reserve 15% of the official training portion as a validation set using random seed 42. The validation split should be stratified by sketch style. The official test set must not be used during training or hyperparameter selection. Resize the photographs and sketches to 128×128 pixels and preserve the correct pairing between every photograph and its target sketch.

The generator should follow a U-Net-style encoder-decoder architecture. It will receive a facial photograph and a selected sketch-style condition and generate a corresponding sketch. The discriminator should follow a PatchGAN design and determine whether local regions of a photograph-sketch pair are real or generated.

The generator can be represented as:

$$\hat{y} = G(x, s)$$

Here, x is the input photograph, s is the selected style condition, and $\hat{y}$ is the generated sketch. The style condition should be represented using a learned categorical embedding for the three FS2K style categories. The embedding must be incorporated into the generator and discriminator rather than being used only as an interface label.

The discriminator receives the photograph, style condition, and either the real or generated sketch:

$$D(x, y, s) \text{ and } D(x, G(x, s), s)$$

The discriminator should be trained to classify real paired sketches as real and generated sketches as fake. The generator should attempt to fool the discriminator while also remaining close to the paired ground-truth sketch.

The generator objective must combine adversarial and reconstruction losses:

$$\mathcal{L}_G = \mathcal{L}_{adv} + \lambda_{L1} \mathcal{L}_{L1}(y, G(x, s))$$

The discriminator and adversarial components may use binary cross-entropy with logits. A reasonable initial reconstruction weight is $\lambda_{L1} = 100$, as commonly used in paired image-to-image translation, but the final value must be investigated through Optuna.

Optuna must be used to tune at least the generator and discriminator learning rates, batch size, base channel count, dropout rate, style-embedding dimension, and reconstruction-loss weight. Because GAN training is computationally demanding, the Optuna trials may use fewer epochs. The selected configuration must then be retrained for the complete training schedule.

Any spatial data augmentation, such as cropping, flipping, rotation, or resizing, must be applied identically to both members of a photograph-sketch pair. Applying unrelated transformations to the two images would destroy the pixel-level correspondence and invalidate the paired training process.

The training process must record the discriminator real loss, discriminator fake loss, generator adversarial loss, generator reconstruction loss, and validation measurements separately. Selected generated samples should be logged after fixed intervals using the same validation photographs so that the development of the generator can be examined over time.

The application must provide a workspace named Face-to-Sketch Generator. A user should be able to upload a facial photograph or capture one using a webcam, select Style 1, Style 2, or Style 3, and generate the corresponding sketch. The interface should display the original photograph and generated sketch side by side and provide an option to download the result. The three fixed style names are valid categorical conditions; unrestricted natural-language prompting is not required.

The final generator must be exported to ONNX. Only the generator is required during application inference; the discriminator remains a training component.

Application and Demonstration Requirements

The final product should appear as one coherent software application rather than four unrelated scripts. The application should contain four clearly accessible workspaces corresponding to the four tasks. The visual design, page structure, colours, controls, cards, image panels, and responsive layout must first be developed in Google Stitch. Evidence of the original Stitch design should be included in the report.

The React frontend must communicate with a FastAPI backend. The backend should validate uploaded files, preprocess the images, load the required ONNX models, perform inference, and return the result with relevant routing and timing information. At minimum, the backend should provide health-check, universal-restoration, hard-routing, soft-mixture, and face-to-sketch operations.

The complete application must run locally through Docker Compose. Public hosting through a suitable service is optional, but a working local containerized deployment is compulsory. If remote hosting is used, the hosted application must remain accessible during the assessment period.

During the live evaluation, previously unseen images may be supplied through the application. The evaluator may select corruption types and severity levels, upload an already corrupted image, inspect classifier and mixture weights, generate sketches, and restart the application from the submitted repository.

Technical Report

Prepare a technical report written in the style of a research paper. It should contain a concise problem introduction, related research, dataset preparation, architecture design, loss functions, training procedure, Optuna search design, experimental setup, results, analysis, application architecture, limitations, and conclusion.

Each of the four tasks must have a clearly identifiable methodology and results discussion. The report must include architecture diagrams, the complete corruption configuration, training and validation curves, Optuna results, confusion matrices where applicable, quantitative result tables, routing-weight visualizations, generated-image grids, error maps, screenshots of the application, and meaningful failure cases.

The report should not consist only of screenshots or raw program output. Every important table, diagram, or image must be interpreted in the accompanying text. You must explain what the result demonstrates, why a failure occurred, and how the evidence influenced your technical conclusions.